EVOLUÇÃO DA WEB — CADERNO INDIVIDUAL DE PERGUNTAS E RESPOSTAS

Nome ou identificação acadêmica: [PREENCHER]

Turma: [PREENCHER]

Data(s): 01/10/2026

Navegador utilizado: [PREENCHER — ex.: Google Chrome / Microsoft Edge + versão]

Ferramenta: Linha_do_Tempo_WEB_Laboratorio_v2.html (2.0.0)

Última etapa concluída / próxima ação: Tarefa 18 concluída / gerar PDF e enviar no Moodle

Este é seu documento principal. Mantenha as perguntas e escreva as respostas abaixo delas. Não escreva no tutorial compartilhado. Registre o que observou; ao revisar uma hipótese, preserve a versão inicial e acrescente a revisão. As situações são simulações locais, não resultados de um pentest real.

Q01 — HTML e hipertexto

> *Pergunta: Qual é a diferença entre Internet, Web, hipertexto e HTML? Use a navegação entre os dois documentos da demonstração para explicar a função de cada conceito.*

A Internet é a infraestrutura de comunicação: redes interligadas que permitem que um computador troque dados com outro. Ela é o caminho, não o conteúdo. A Web é um serviço que roda sobre esse caminho: um conjunto de documentos e recursos identificados por endereços, obtidos por um protocolo (HTTP) e representados em um formato comum. Hipertexto é a ideia de texto com ligações, em que um documento aponta diretamente para outro. HTML é a linguagem de marcação que escreve esse hipertexto, ou seja, que diz ao navegador "este trecho é um título", "este trecho é um link para tal documento".

Na demonstração isso ficou visível: no Documento A ("Introdução à Web"), em Sem o recurso, só havia a referência textual "O que é uma rede?" e eu teria que localizar esse documento por conta própria. Em Com o recurso, apareceu o link "Leia também: o que é uma rede?"; ao clicar, fui para o Documento B e pude voltar pela ligação "Voltar à introdução". A relação entre os dois documentos é o hipertexto; o que a escreveu foi o HTML; o conjunto de documentos ligados é a Web; e a Internet seria o meio por onde os documentos trafegariam se estivessem em servidores diferentes (na ferramenta a passagem é local, nenhum servidor externo é acessado).

Q02 — HTML e hipertexto

> *Pergunta: Sem HTML — nem outro padrão equivalente que cumprisse essa função — o que seria mais difícil no acesso a informações de diferentes fontes? Explique por que isso não significa que a Internet deixaria de existir.*

Sem um padrão comum de representação e de ligação, cada fonte publicaria seus documentos em um formato próprio e eu precisaria de um programa diferente (ou de instruções específicas) para ler cada uma. Além disso, uma referência a outro documento seria só um texto, como no modo Sem o recurso da demonstração: eu teria de descobrir onde o documento está, como buscá-lo e como abri-lo. O que ficaria

difícil é justamente relacionar informações de origens diferentes e passar de uma para outra com um único gesto, em qualquer navegador.

Isso não acabaria com a Internet porque ela é a camada de comunicação e é anterior e independente da Web. E-mail, transferência de arquivos e acesso remoto, por exemplo, usam a mesma rede sem depender de HTML. Faltaria a camada que organiza e conecta documentos, não a que transporta os dados.

Q03 — Formulários — benefício

> *Pergunta: Que informação você enviou na busca e que resultado recebeu? Compare Sem o recurso e Com o recurso. O que essa capacidade acrescentou à experiência de consultar documentos?*

Em Com o recurso, digitei "redes" no campo Assunto e cliquei em Buscar: o portal respondeu "1 material encontrado" — Redes de computadores (Equipe da biblioteca). Com "web" vieram 3 materiais (Hipertexto e a construção da Web, Segurança de aplicações e Mensagens HTTP) e com "http" veio 1 material (Mensagens HTTP). A resposta normal traz somente título e autor de materiais do catálogo público.

Em Sem o recurso, o portal mostra apenas o texto "Catálogo da biblioteca — Redes · Sistemas · Segurança", sem nenhum campo: eu só posso ler o que já foi publicado e seguir referências.

O formulário acrescentou a possibilidade de eu enviar um dado meu (o assunto) para o servidor e receber uma resposta montada para aquela pergunta. A Web deixou de ser só leitura de documentos prontos e passou a aceitar entradas do usuário, sem que eu precise conhecer o caminho de cada documento.

Q04 — Formulários — observação e hipótese

> *Pergunta: O que mudou entre o resultado normal e o comportamento inesperado? Registre primeiro a mudança observável, depois sua hipótese inicial e uma observação que poderia contrariá-la. Referencie E01.*

Mudança observável (E01): sem eu ter feito uma nova busca, a área de resultado deixou de mostrar materiais do catálogo e passou a exibir um bloco "Resultado inesperado · cadastros fictícios" com três leitores: Ana · leitor 014 (ana@aluno.invalid), Bruno · leitor 027 (bruno@aluno.invalid) e Carla · leitor 039 (carla@aluno.invalid). O formulário continuou igual; o que mudou foi o tipo de dado devolvido, que não é título nem autor.

Hipótese inicial: uma busca diferente da minha chegou ao servidor e o texto enviado no campo Assunto foi tratado como parte do comando de consulta, e não apenas como palavra a procurar. Por isso o servidor leu outra tabela (cadastros) além do catálogo.

O que poderia contrariar a hipótese: se o percurso mostrasse que a consulta do servidor é fixa e o valor nunca entra no texto do comando, ou se os cadastros aparecessem mesmo sem nenhuma requisição nova (por exemplo, por um erro de permissão na página), a explicação teria de ser outra — falha de autorização, e não de interpretação da entrada.

Revisão da hipótese (depois de Rastrear): a hipótese se manteve. O modelo confirma uma requisição adversária predefinida e uma consulta montada por concatenação. Acrescento uma condição que eu não tinha considerado: o efeito também depende de a conta do banco ter permissão de leitura sobre os cadastros.

Evidência E01: ver índice de evidências.

Q05 — Formulários — caminho dos dados

> *Pergunta: Reconstrua o caminho da entrada até a resposta. Quem transporta os dados, quem constrói a consulta e quem interpreta SQL no sistema representado? Em qual passagem a entrada deveria permanecer separada do comando?*

1) Cliente: o campo do formulário vira um parâmetro. O cliente controla os bytes enviados, mesmo sem usar a interface esperada. 2) Requisição: o HTTP transporta o dado (GET /catalogo?assunto=...). O protocolo só carrega; ele não executa SQL. 3) Aplicação no servidor: é ela quem constrói a consulta. Na versão vulnerável, o pseudocódigo junta o texto "SELECT titulo, autor FROM catalogo WHERE assunto = '" com a entrada, ou seja, o valor participa da construção do comando. 4) Banco de dados: é quem interpreta SQL e devolve os registros; a resposta indevida trouxe dados fora do contrato da consulta.

A separação deveria existir na passagem 3, entre a aplicação e o banco: o comando precisa ser fixo e a entrada precisa chegar ao banco como valor vinculado (parâmetro), nunca como pedaço da instrução. O limite de confiança é "entrada do cliente → código da aplicação → instrução interpretada pelo banco".

Q06 — Formulários — decisões e retestes

> *Pergunta: Para cada uma das quatro intervenções, registre sua previsão antes do teste e os dois resultados depois: o efeito foi contido e a busca legítima continuou disponível? Explique por que pelo menos uma alternativa não tratou o mecanismo e por que outra eliminou também o benefício.*

Intervenção 1 — Adicionar HTTPS / Previsão: não deve resolver, porque protege o trajeto e não a forma como o servidor monta a consulta; a busca normal deve continuar. / Entrada adversária: efeito ainda ocorre — a resposta continua contendo os três cadastros, só que agora poderia trafegar por canal protegido. / Uso legítimo: preservado, a busca continua disponível. / Explicação: a previsão se confirmou. O HTTPS atua na camada de transporte; a concatenação no servidor permaneceu igual.

Intervenção 2 — Limitar o campo a 20 caracteres / Previsão: achei que poderia atrapalhar uma entrada longa, mas que não seria confiável, porque o limite fica no navegador; busca normal mantida. / Entrada adversária: efeito ainda ocorre — a requisição do modelo não depende do limite visual e os registros continuam expostos. / Uso legítimo: preservado, com uma restrição visual a mais. / Explicação: a restrição está na interface, e a interface não controla todas as requisições possíveis. O servidor continuou concatenando.

Intervenção 3 — Parametrizar a consulta no servidor / Previsão: deve conter o efeito e manter a busca, porque atua exatamente no ponto em que a entrada entrava no comando. / Entrada adversária: efeito impedido — o marcador permanece um valor de busca e nenhum cadastro privado é retornado. / Uso legítimo: preservado — a busca por "redes" continua retornando o material público. / Explicação: previsão confirmada. O parâmetro é tratado como valor e não altera a estrutura da instrução. A própria ferramenta avisa que isso não substitui autorização nem menor privilégio.

Intervenção 4 — Desativar o serviço de busca / Previsão: impede o efeito, mas ao custo de perder a busca. / Entrada adversária: efeito impedido — a operação não é processada porque o serviço foi desativado. / Uso legítimo: perdido — o leitor não consegue mais consultar o catálogo; a tela mostra "Serviço de busca desativado pelo controle escolhido". / Explicação: previsão confirmada. O caminho deixou de existir para todos.

Por que duas não trataram o mecanismo: HTTPS e limite de caracteres agem em outras camadas (canal e interface). O problema está na aplicação, quando transforma dado em sintaxe SQL. Por que uma eliminou

o benefício: desativar a busca remove a entrada adversária e a legítima ao mesmo tempo; bloquear tudo não é o mesmo que manter a função com segurança.

Q07 — Formulários — conclusão e transferência

> *Pergunta: Qual intervenção tratou o problema preservando a busca, e qual evidência do reteste sustenta sua conclusão? Referencie E02. Uma API que recebe o assunto em JSON precisaria do mesmo cuidado? Declare também o que este modelo não permite comprovar sobre um sistema real.*

A parametrização da consulta no servidor. A evidência é o reteste registrado em E02: no cartão "Entrada adversária simulada" apareceu "Efeito impedido no modelo" e, no cartão "Uso legítimo", "Benefício preservado"; na demonstração a faixa mostrou "Efeito contido; benefício preservado" e, ao usar Testar uso legítimo, a busca por "redes" voltou a trazer 1 material público. Nenhuma das outras três alternativas apresentou os dois resultados positivos ao mesmo tempo.

Uma API que recebe o assunto em JSON precisa do mesmo cuidado. O JSON muda apenas a embalagem do dado no transporte; se o servidor pegar o campo "assunto" e concatená-lo ao comando, o ponto vulnerável é idêntico. O que importa é como o valor entra na consulta, não se veio de formulário, URL ou JSON.

Limites: o laboratório não tem banco nem executa SQL; os resultados são predefinidos. Ele não comprova que um sistema real está seguro, não mostra o código verdadeiro de uma aplicação, não testa outros pontos de entrada e pressupõe uma conta de banco com acesso aos cadastros. Num sistema real eu ainda precisaria verificar autorização, privilégios da conta de banco e validação de entrada.

Evidência E02: ver índice de evidências.

Q08 — Cookies — benefício

> *Pergunta: O que ocorreu com a reserva ao mudar de página sem o recurso e com o recurso? Diferencie o mecanismo de cookie do contexto de sessão, sem afirmar que cookies são a única maneira possível de manter continuidade.*

Sem o recurso, reservei um material e, ao clicar em Ir a outra página, o modelo perdeu a associação: a nova página não reconheceu a reserva que eu tinha feito. A tela avisa que, nesse exemplo, nenhuma alternativa ao cookie associa a sessão entre páginas. Com o recurso, reservei, mudei de página e o portal continuou mostrando "Sua reserva continua aqui", com a sessão fictícia visita-A e o local de retirada Biblioteca central.

O cookie é o mecanismo: um pequeno dado que o servidor entrega ao navegador e que o navegador devolve nas requisições seguintes. A sessão é o contexto: o conjunto de informações (quem é a visita, quais reservas tem) que o servidor mantém e associa àquele identificador. O HTTP não guarda memória entre requisições, então algo precisa carregar o identificador. O cookie é a forma mais comum, mas não a única: o identificador poderia ir na URL, em um campo oculto de formulário ou em um cabeçalho com token. O laboratório só não implementou essas alternativas.

Q09 — Cookies — observação e hipótese

> *Pergunta: Que dado da reserva mudou sem uma escolha sua? A sessão continuou reconhecida? Registre*

> *a observação, sua hipótese inicial e o que a faria ser revista. Referencie E03.*

Observação (E03): o local de retirada mudou de Biblioteca central para Balcão externo sem que eu clicasse em nada. A tela passou a mostrar o selo "Alteração inesperada" e a comparação "Você escolheu: Biblioteca central / Servidor registrou: Balcão externo". A sessão continuou reconhecida: o identificador visita-A e a quantidade de materiais permaneceram.

Hipótese inicial: alguém ou alguma outra página enviou ao servidor um pedido de alteração em meu nome, e o servidor aceitou porque o pedido chegou junto com a minha sessão. Ou seja, o servidor reconheceu quem eu era, mas não conferiu se a operação tinha sido realmente iniciada por mim.

O que me faria rever: se a sessão tivesse caído ou mudado de identificador (o que indicaria roubo ou troca de sessão), ou se o rastreamento mostrasse que a alteração partiu da própria página do portal por um erro interno, a explicação seria diferente.

Revisão da hipótese (depois de Rastrear): confirmada. O modelo mostra uma origem externa induzindo a requisição, com o cookie enviado automaticamente e sem token anti-CSRF.

Evidência E03: ver índice de evidências.

Q10 — Cookies — caminho dos dados

> *Pergunta: No cenário representado, de onde parte a operação, como a credencial a acompanha e qual verificação falta antes da alteração? O mecanismo pressupõe roubo da senha? Explique as condições do modelo, sem generalizar o envio de cookies a qualquer requisição.*

A operação parte de outro contexto, externo ao portal, que induz o navegador a fazer a requisição (POST /reserva/retirada, local=balcao_externo, token ausente). A credencial acompanha porque o navegador simulado anexa o cookie de sessão automaticamente, já que as condições e políticas do modelo permitem esse envio. A aplicação valida a sessão e confirma que a reserva pertence ao usuário, mas falta verificar o token anti-CSRF, isto é, a prova de que o pedido saiu de um formulário legítimo da própria aplicação. Sem essa verificação, o servidor altera o local.

O mecanismo não pressupõe roubo de senha nem do identificador de sessão: a origem externa nem precisa conhecer o cookie, basta que o navegador o envie.

Condições do modelo: a requisição precisa ser elegível para envio de cookies, as políticas (como SameSite) precisam permitir, e a aplicação precisa não ter token nem validação equivalente. Em outras configurações o navegador não enviaria o cookie em uma requisição vinda de outro site, então não é correto dizer que qualquer requisição externa carrega qualquer cookie. No laboratório não há cookie real nem requisição entre sites.

Q11 — Cookies — decisões e retestes

> *Pergunta: Compare as quatro intervenções do painel Proteger. Para cada uma, escreva previsão, resultado para a operação indevida e resultado para o uso legítimo. Por que identificar a sessão, mudar o método HTTP ou impedir a leitura do cookie não resolve necessariamente o mesmo problema?*

Intervenção 1 — Usar apenas HttpOnly no cookie / Previsão: não deve impedir, porque o ataque do modelo não lê o cookie, só aproveita o envio automático; uso legítimo mantido. / Operação indevida: efeito ainda ocorre — a credencial continua acompanhando a operação simulada e o local é alterado. / Uso legítimo: preservado, a reserva continua disponível. / Explicação: HttpOnly impede a leitura por

JavaScript, o que ajuda contra roubo de sessão via script, mas não muda o envio do cookie nas requisições.

Intervenção 2 — Trocar GET por POST, sem outro controle / Previsão: fiquei em dúvida; imaginei que dificultaria, mas que uma página externa também consegue enviar POST. / Operação indevida: efeito ainda ocorre — a operação POST é aceita sem token anti-CSRF válido. / Uso legítimo: preservado. / Explicação: usar o método adequado é boa prática, mas o método não prova a origem nem a intenção do pedido.

Intervenção 3 — Desativar alterações de reservas / Previsão: impede a alteração indevida, mas tira a função de todos. / Operação indevida: efeito impedido — a reserva não muda porque o serviço de alteração está desligado. / Uso legítimo: perdido — o usuário não consegue corrigir uma retirada legítima; a tela mostra "Alterações de reservas desativadas". / Explicação: previsão confirmada; segurança obtida eliminando o benefício.

Intervenção 4 — Validar token anti-CSRF no servidor / Previsão: deve rejeitar o pedido externo, que não tem o token, e aceitar o pedido feito pelo próprio portal. / Operação indevida: efeito impedido — requisição rejeitada com 403 no modelo; Biblioteca central é preservada. / Uso legítimo: preservado — a alteração legítima, acompanhada do token válido, continua funcionando. / Explicação: previsão confirmada. O servidor passa a exigir algo que o contexto externo não possui.

Por que não são o mesmo problema: identificar a sessão responde "quem é o usuário"; não responde "foi ele quem pediu isto". Mudar o método HTTP altera a forma do pedido, não sua legitimidade. Impedir a leitura do cookie protege a confidencialidade da credencial, mas aqui a credencial não é lida, é apenas usada. O defeito está em confundir sessão válida com operação legítima.

Q12 — Cookies — conclusão e transferência

> *Pergunta: Qual intervenção preservou a função legítima e impediu a alteração indevida nas condições do modelo? Referencie E04. Aplique o raciocínio a uma troca de e-mail autenticada por cookie e explique por que esse controle não substitui autorização.*

A validação do token anti-CSRF no servidor. Em E04, o reteste mostrou "Efeito impedido no modelo" (requisição rejeitada com 403, local mantido em Biblioteca central) e "Benefício preservado" (a alteração legítima com token válido continua funcionando). Ao usar Testar uso legítimo, a reserva seguiu reconhecida na sessão visita-A com retirada na Biblioteca central.

Troca de e-mail autenticada por cookie: é o mesmo desenho. Se o endpoint aceitar a mudança só porque o cookie de sessão chegou, uma página externa pode induzir o navegador a trocar o e-mail de contato da vítima — e isso costuma abrir caminho para a recuperação de senha. O servidor deve exigir um token anti-CSRF válido e associado à sessão antes de alterar, e, por ser uma operação sensível, faz sentido pedir a senha atual ou outra confirmação.

Por que não substitui autorização: o token prova que o pedido saiu de uma página legítima daquela sessão; não diz se aquele usuário tem direito sobre o recurso. Um usuário autenticado, com token válido, ainda poderia tentar alterar o e-mail ou a reserva de outra conta trocando um identificador. Autenticar (quem é), autorizar (o que pode) e verificar a intenção do pedido (anti-CSRF) são três verificações diferentes, e o servidor precisa das três. SameSite adequado e proteção contra XSS também continuam necessários.

Evidência E04: ver índice de evidências.

Q13 — HTTPS

> *Pergunta: Compare o que um observador do trecho de rede consegue ler no modelo com e sem HTTPS. Qual benefício a proteção oferece e que decisão sobre acesso a dados ela não toma pela aplicação?*

Sem o recurso, ao enviar o pedido, o observador do trecho leu tudo em texto: "GET /pedido?id=42" e "Resposta: pedido de Ana". Com o recurso, o mesmo observador viu apenas "[registros cifrados — conteúdo HTTP não legível]". O benefício é proteger o conteúdo em trânsito contra leitura (e também contra alteração) por quem está no meio do caminho, além de validar com quem o navegador está falando.

O que o HTTPS não decide: se quem pediu o pedido 42 tem direito de ver o pedido de Ana. Nas pontas do canal a aplicação acessa o conteúdo normalmente; se o servidor entregar o pedido de outra pessoa a qualquer usuário que troque o número do id, ele fará isso por um canal perfeitamente cifrado. A decisão de acesso é da aplicação. Observação: a ferramenta apenas representa o canal, não implementa criptografia real, e as menções a SSL são históricas, não recomendação de configuração.

Q14 — Java e JavaScript

> *Pergunta: Qual é a diferença entre o applet Java representado no simulador e o código JavaScript executado pela página? Identifique os ambientes de execução e uma responsabilidade de segurança de cada um, sem tratar applets como programas com acesso irrestrito por padrão.*

Apesar do nome parecido, são tecnologias distintas. O applet Java era um programa compilado que a página incorporava e que rodava dentro de uma JVM (máquina virtual Java), um componente adicional instalado na máquina do usuário por meio de plug-in. No simulador, sem o recurso o applet simplesmente não executa ("Sem o ambiente Java, este applet não executa"); com o recurso, o pêndulo anima. O JavaScript é uma linguagem interpretada pelo motor do próprio navegador, sem instalação extra, e atua diretamente sobre a página — foi assim que o subtotal da tarefa 06 foi calculado.

Responsabilidades: os applets não tinham acesso irrestrito por padrão; rodavam em uma área restrita (sandbox) da JVM. A responsabilidade era manter esse isolamento funcionando, o que exigia que o usuário mantivesse a JVM/plug-in atualizado, pois uma falha nesse componente comprometia a restrição. No JavaScript, o isolamento é do navegador (cada página fica limitada à sua origem) e a responsabilidade principal da aplicação é não tratar como confiável o que é calculado ou validado no cliente e não deixar dados não confiáveis virarem código da página.

Limite: a animação do laboratório é feita com JavaScript/CSS; nenhum Java foi instalado ou executado.

Q15 — JavaScript

> *Pergunta: Qual subtotal você obteve para três cadernos de R$ 25,00 e onde ele foi calculado no modo Com o recurso? Por que um servidor de compras não deve aceitar esse subtotal como autoridade apenas porque veio da interface?*

Obtive R$ 75,00 (3 × R$ 25,00). No modo Com o recurso, a mensagem foi "Resultado calculado no navegador, sem requisição", ou seja, a conta foi feita no cliente. Em Sem o recurso, o campo mostrou "Depende do servidor", indicando que seria necessária uma requisição.

O servidor não deve aceitar esse valor como autoridade porque tudo o que roda no navegador está sob controle do usuário: ele pode alterar o script, mudar o valor pelas ferramentas de desenvolvedor ou enviar a requisição diretamente, sem passar pela página, com o subtotal que quiser. O cálculo no cliente serve para dar resposta rápida; a decisão de negócio (preço, quantidade, total a cobrar) precisa ser refeita no servidor com os preços que ele mesmo guarda.

Q16 — CSS

> *Pergunta: Ao trocar a folha de estilo, o que mudou e o que permaneceu? Esconder um botão remove a permissão de executar a operação correspondente? Justifique separando apresentação, requisição e decisão de acesso.*

Ao clicar em Trocar a folha de estilo, mudaram cores, fundo e a aparência geral do bloco. Permaneceram o conteúdo e a estrutura: "Biblioteca de redes", "Documentos, projetos e referências conectados" e "3 coleções · 12 documentos" continuaram iguais; a própria tela indica "A mesma estrutura HTML". Mesmo sem a folha CSS do exemplo, o navegador aplica uma apresentação padrão.

Esconder um botão não remove a permissão. Apresentação: o CSS só decide como (e se) algo aparece na tela; o elemento continua no documento. Requisição: a operação é disparada por uma requisição HTTP, que pode ser montada sem o botão — reexibindo o elemento, editando a página ou enviando o pedido diretamente. Decisão de acesso: é tomada no servidor, que deve verificar a cada requisição se aquele usuário pode executar a operação. Se essa verificação não existir, o botão escondido é só uma dificuldade visual.

Q17 — Flash e plug-ins

> *Pergunta: Que benefício o player adicional representava e que componente extra precisava ser mantido? Uma atualização somente no servidor corrigiria automaticamente um problema no plug-in do usuário? Explique.*

O player trazia animação e conteúdo multimídia interativo que o navegador da época não oferecia sozinho. Com o recurso, a animação reproduziu; sem o recurso, apareceu "Player adicional não disponível" — e a tela lembra que isso não impedia outras interações na Web, só o conteúdo que dependia do player.

O componente extra era o próprio plug-in, instalado na máquina de cada usuário, com ciclo de atualização separado do navegador e do site. Por isso uma atualização só no servidor não corrige uma falha no plug-in: o servidor atualiza o conteúdo que envia, mas o código defeituoso está no computador do visitante e continua lá até que cada usuário (ou o fabricante) atualize ou remova o player. É um componente fora do alcance de quem mantém o site.

Q18 — DOM — benefício

> *Pergunta: Que comentário você publicou e como ele passou a aparecer? Compare o documento inicial com a atualização local. Qual é o benefício de modificar o DOM sem carregar uma nova página?*

Publiquei "O material sobre hipertexto ajudou a entender a diferença entre Web e Internet." e depois "Gostaria de mais exemplos sobre requisições HTTP." [AJUSTAR se você publicar outras frases]. Cada comentário apareceu na hora no mural "Converse sobre o material", abaixo do comentário do leitor fictício, e o contador passou para "2 contribuições locais", sem a página piscar ou recarregar.

O documento inicial (Sem o recurso) só mostra "Comentários sobre o material" com a pergunta "Qual material ajuda a entender hipertexto?" e nenhuma forma de atualização local. Com o recurso, o script altera a árvore do documento que está na memória do navegador, acrescentando um elemento novo.

O benefício é a resposta imediata: só o trecho necessário muda, o restante da página e o que eu estava fazendo são preservados e não é preciso pedir e redesenhar o documento inteiro.

Q19 — DOM — observação e hipótese

> *Pergunta: O que mudou na aparência e na origem aparente da mensagem? Registre sua hipótese antes de ler a explicação. O efeito observado comprova execução de JavaScript ou apenas interpretação de marcação no modelo? Referencie E05 e diga qual evidência faltaria para uma conclusão mais forte.*

Observação (E05): primeiro surgiu no mural um comentário de "Visitante simulado · nova entrada" com o texto AVISO NÃO OFICIAL aparecendo com as marcas. Em seguida, o título do mural trocou de "Converse sobre o material" para "Quem está falando nesta página?", o comentário passou a aparecer só como AVISO NÃO OFICIAL (sem as marcas) e surgiu uma faixa destacada "ESTE AVISO NÃO VEIO DA BIBLIOTECA — Uma entrada ganhou aparência de interface no modelo". A caixa para eu comentar sumiu enquanto o efeito estava ativo. A origem aparente mudou: um conteúdo de visitante passou a parecer um aviso do próprio portal.

Hipótese inicial: o comentário do visitante continha marcação HTML e a página, ao inseri-lo, tratou esse conteúdo como parte da estrutura do documento em vez de mostrá-lo como texto.

Inferência: o que observei comprova apenas interpretação de marcação no modelo (uma alteração de aparência). Não comprova execução de JavaScript. Para uma conclusão mais forte de XSS seria preciso evidência de um script efetivamente executado — por exemplo, uma ação disparada pelo conteúdo inserido, um registro no console ou uma requisição gerada por ele — observada em uma aplicação real, não em um efeito programado.

Revisão da hipótese (depois de Rastrear): mantida; o pseudocódigo mostra caixa.innerHTML = comentario. [Se executar a extensão opcional "Simular um possível efeito de XSS": a faixa acrescenta "Extensão de XSS simulada: prioridade local alterada para ALTA. Nenhum script de visitante foi executado." É uma segunda simulação predefinida, não prova de código malicioso executado.]

Evidência E05: ver índice de evidências.

Q20 — DOM — caminho dos dados

> *Pergunta: Identifique a fonte da entrada, a operação que a recebe e o componente que interpreta o resultado. Qual diferença existe entre inserir texto e interpretar HTML? Por que a análise não pode se limitar à resposta original do servidor?*

Fonte: o comentário de um visitante, que pode vir de uma caixa de texto, da URL ou da resposta de uma API; a origem não o torna confiável. Operação que recebe: o código cliente (JavaScript da página), que escolhe como colocar o comentário no documento — no caso vulnerável, com innerHTML. Componente que interpreta: o navegador, ao processar aquela atribuição e transformar a entrada em elementos do DOM. O efeito aparece na interface.

Inserir texto (textContent) faz o navegador mostrar os caracteres exatamente como vieram, inclusive os sinais < e >; a entrada continua sendo dado. Interpretar HTML (innerHTML) faz o navegador analisar a

entrada como marcação e criar elementos; a entrada passa a ser estrutura.

A análise não pode parar na resposta original do servidor porque essa resposta pode estar correta e segura, e o problema surgir depois, quando o script do cliente altera o documento em memória. O HTML que saiu do servidor e o DOM final não são a mesma coisa; é preciso examinar o código do cliente e o estado final do documento.

Q21 — DOM — decisões e retestes

> *Pergunta: Teste as quatro intervenções. Para cada uma, registre previsão, efeito sobre a entrada adversária e continuidade do comentário legítimo. Por que comprimento da entrada, proteção do transporte e contexto de interpretação não são a mesma coisa?*

Intervenção 1 — Limitar comentários a 200 caracteres / Previsão: não deve resolver, porque a marcação usada é curta e cabe no limite; comentário normal mantido. / Entrada adversária: efeito ainda ocorre — o aviso indevido continua aparecendo no modelo. / Uso legítimo: preservado, comentários curtos continuam funcionando. / Explicação: o tamanho não muda a operação que interpreta a entrada.

Intervenção 2 — Inserir o comentário com textContent / Previsão: deve conter o efeito e manter os comentários, pois trata a entrada como texto, que é o requisito do mural. / Entrada adversária: efeito impedido — a marcação aparece literalmente e não cria aviso nem comportamento; a tela mostra "A mesma entrada, agora literal: AVISO NÃO OFICIAL". / Uso legítimo: preservado — o novo comentário aparece imediatamente e sem recarga. / Explicação: previsão confirmada. A ferramenta ressalva que isso não é receita para todos os contextos (HTML, JS, URL).

Intervenção 3 — Adicionar somente HTTPS / Previsão: não resolve; o problema acontece depois que o dado já chegou ao navegador. / Entrada adversária: efeito ainda ocorre — o conteúdo ainda é interpretado como parte da interface. / Uso legítimo: preservado. / Explicação: o canal protegido não altera a operação no DOM.

Intervenção 4 — Desativar comentários por completo / Previsão: impede o aviso, mas elimina a participação do leitor. / Entrada adversária: efeito impedido — o aviso não entra porque nenhum comentário é aceito. / Uso legítimo: perdido — não é possível publicar; a tela mostra "Comentários desativados pelo controle escolhido". / Explicação: previsão confirmada; o benefício foi removido junto.

Por que não são a mesma coisa: comprimento é uma propriedade da quantidade de dados; transporte diz respeito a quem pode ler ou alterar os dados no caminho; contexto de interpretação diz respeito ao que o navegador faz com os dados ao recebê-los. Uma entrada curta, entregue por canal cifrado, ainda vira estrutura se for colocada em uma operação que interpreta HTML.

Q22 — DOM — conclusão e transferência

> *Pergunta: Que intervenção atendeu ao requisito de comentários em texto simples sem remover a funcionalidade? Publique outro comentário, referencie E06 e explique o resultado. Receber o comentário em JSON de uma API o tornaria automaticamente seguro para uma operação que interpreta HTML? Declare os limites da simulação.*

Inserir o comentário com textContent. Em E06, o reteste mostrou "Efeito impedido no modelo" e "Benefício preservado": a marcação do visitante apareceu de forma literal, sem criar aviso, e a caixa de comentário voltou a ficar disponível. Publiquei o novo comentário "Concluí que o comentário deve ser

inserido como texto, não como HTML." [AJUSTAR se publicar outra frase] e ele apareceu imediatamente no mural, sem recarga e sem alterar a aparência do portal.

Receber o comentário em JSON não o torna seguro. JSON é só o formato de transporte; depois de decodificado, o campo é uma cadeia de caracteres como qualquer outra. Se o cliente colocar essa cadeia em innerHTML, ela será interpretada como marcação do mesmo jeito. O que define o risco é o contexto em que o dado é inserido, não a forma como chegou.

Limites: todo o efeito é programado e local; nenhum código digitado por mim foi executado; a mudança visual é uma simulação de injeção de HTML via DOM e não uma prova de execução de JavaScript. O resultado vale para o requisito de texto simples. Se a aplicação precisasse aceitar HTML nos comentários, seriam necessárias sanitização adequada e, como camada adicional, CSP.

Evidência E06: ver índice de evidências.

Q23 — XHR / AJAX

> *Pergunta: Compare a busca com atualização parcial com o modelo de recarga. Que trabalho continua no servidor e por que remover o botão da interface não elimina a obrigação de autorizar o acesso ao endpoint?*

Com o recurso, ao buscar um assunto, o cartão "Pesquisa no catálogo" permaneceu na tela (rotulado "Interface preservada"), o botão mostrou "Consultando…" por um instante e apareceu "Somente este resultado mudou", com o resultado demonstrativo. Sem o recurso, o rótulo passou a "Modelo de navegação com recarga" e o resultado veio com "Uma nova página seria carregada": todo o documento seria substituído para mostrar a mesma informação.

O trabalho do servidor é igual nos dois casos: receber a requisição, verificar quem pede e se pode, consultar os dados e montar a resposta. O que muda é só o que o navegador faz com ela.

Remover o botão não elimina a obrigação de autorizar porque o endpoint continua existindo e respondendo a qualquer requisição que chegue no endereço certo. Como a chamada é feita por script, ela pode ser reproduzida fora da página. A interface apenas facilita o acesso; quem controla o acesso é a verificação feita no servidor a cada requisição.

Limite: a busca é simulada localmente; o painel Network não mostraria chamada real.

Q24 — WebSocket

> *Pergunta: Qual diferença você observou entre consultar repetidamente e receber eventos em um canal aberto? Se uma permissão é retirada depois da abertura, por que a conexão não deve conservar automaticamente aquele poder?*

Sem o recurso, cada novidade só apareceu quando eu cliquei em Consultar novamente ("Resposta à nova consulta: atualização N disponível") — o cliente precisa perguntar de tempos em tempos. Com o recurso, ao gerar um evento no servidor, a mensagem "Servidor envia: atualização N disponível" chegou pelo "canal simulado aberto nos dois sentidos", sem um novo pedido do cliente.

A conexão dura muito mais que uma requisição comum. Se a permissão for verificada apenas na abertura, um usuário que perdeu o acesso (saiu da turma, teve a conta bloqueada, encerrou a sessão) continuaria recebendo e enviando mensagens enquanto o canal estivesse aberto. Canal aberto prova que houve

permissão no passado, não que ela ainda vale. O servidor deve verificar a autorização a cada mensagem ou operação sensível e encerrar conexões cuja permissão foi revogada.

Limite: nenhuma conexão WebSocket real é aberta na ferramenta.

Q25 — HTML5 e APIs nativas

> *Pergunta: Qual recurso nativo você experimentou e que dependência adicional ele pode dispensar? Por que a aplicação ainda precisa cuidar de entradas, mensagens e dados locais, mesmo usando APIs fornecidas pelo navegador?*

Experimentei o Canvas: ao clicar em Desenhar no canvas, o próprio navegador desenhou uma curva e um ponto na área gráfica, e a nota informa "Canvas real, desenhado localmente. Não exige plug-in". Sem o recurso, a área mostrou "Este exemplo não usa uma API nativa de desenho". Esse tipo de recurso pode dispensar um player adicional, como o Flash, para gráficos e animação.

Ser nativo significa que a capacidade faz parte do navegador e é mantida com ele; não significa que a aplicação fica correta sozinha. As APIs são ferramentas: a aplicação continua recebendo entradas não confiáveis, trocando mensagens entre janelas e origens (cuja origem precisa ser conferida) e podendo gravar dados no armazenamento local (que qualquer script da página consegue ler, portanto não é lugar para segredos). Quem decide o que fazer com esses dados é o código da aplicação.

Precisão histórica: 2014 é a data de padronização do HTML5, não o lançamento simultâneo de todas as APIs em todos os navegadores.

Q26 — Service Workers

> *Pergunta: O que aconteceu ao simular a falta de conexão com e sem o recurso? Uma página disponível offline garante conteúdo atualizado? Explique o cuidado necessário com respostas privadas e registre se esta ferramenta realmente registrou um service worker ou apenas modelou seu papel.*

Com o recurso, ao simular ficar offline, o material "Aula de redes" continuou disponível com a mensagem "Cópia previamente armazenada entregue pelo cache do modelo". Sem o recurso, a mesma ação resultou em "Conteúdo não disponível — Sem rede e sem cópia adequada neste exemplo".

Disponível offline não é o mesmo que atualizado: o que é exibido é uma cópia feita em algum momento anterior, e o conteúdo no servidor pode ter mudado depois. A aplicação precisa de uma estratégia de atualização e, de preferência, deve avisar quando está mostrando uma cópia.

Respostas privadas: o cache guarda cópias no dispositivo. Se respostas com dados pessoais ou dependentes de login forem armazenadas, podem ser exibidas a outra pessoa que use o mesmo navegador ou continuar acessíveis depois do logout. É preciso escolher o que pode ser armazenado e limpar o cache ao encerrar a sessão.

A ferramenta não registrou um service worker de verdade; a própria tela informa "Nenhum service worker é registrado. O estado é apenas representado em memória". Ela apenas modelou o papel, e eu não desliguei a rede real.

Q27 — WebAssembly

> *Pergunta: Some 19 e 23 nos modos com e sem WebAssembly. Que diferença existe no caminho de execução e que resultado permanece igual? Esse experimento permite afirmar que WebAssembly é sempre mais rápido ou elimina erros de autorização? Justifique.*

Com o recurso, a mensagem foi "O módulo WebAssembly calculou 19 + 23 = 42."; o caminho indicado era "JavaScript → módulo .wasm local → resultado". Sem o recurso, a mensagem foi "JavaScript calculou 19 + 23 = 42.". O resultado, 42, é o mesmo. A diferença está em quem executou a soma: num caso o JavaScript carregou um módulo compilado e chamou a função dele; no outro, o próprio JavaScript fez a conta. [Se no seu navegador aparecer "Módulo não executado: ...", registre essa mensagem aqui e em Limitações.]

O experimento não permite afirmar que WebAssembly é sempre mais rápido: é uma única soma, sem medição de tempo, e a ferramenta declara que não é um benchmark. O ganho depende do tipo de tarefa. Também não elimina erros de autorização: o módulo roda no cliente, dentro do mesmo isolamento do navegador, e a decisão de quem pode acessar o quê continua sendo do servidor. Código compilado não é sinônimo de código confiável.

Q28 — WebAuthn

> *Pergunta: O que muda no modelo de autenticação por chave pública em relação à senha reutilizável? Por que uma autenticação forte não corrige, sozinha, uma consulta insegura, uma interpretação indevida de conteúdo ou uma autorização ausente?*

No modelo de senha (Sem o recurso), o usuário envia um segredo reutilizável e "o servidor verifica a senha enviada por canal protegido": o segredo sai do dispositivo, pode ser digitado em um site falso, reaproveitado em outros serviços ou vazado de onde está guardado. No modelo de chave pública (Com o recurso), o site envia um desafio, o autenticador assina com a chave privada e o servidor confere com a chave pública: "prova validada para o serviço. A chave privada não é enviada ao servidor". O servidor guarda apenas a chave pública, que não serve para se passar pelo usuário, e a credencial é vinculada àquele site, o que dificulta o phishing.

Autenticação forte responde somente à pergunta "quem é você?". Um usuário perfeitamente autenticado ainda pode enviar uma entrada que o servidor concatena em SQL (INV-01), ter o navegador usado para um pedido que não fez (INV-02), ver um comentário interpretado como HTML (INV-03) ou pedir um recurso que não é dele se ninguém verificar a permissão. São controles de camadas diferentes; um não substitui o outro.

Limite: a simulação não usou biometria, chave física nem credenciais reais.

Q29 — Fim do Flash

> *Pergunta: Que benefício pode ser mantido quando um componente legado é removido e a função é migrada? Explique por que evoluir também pode reduzir superfícies de ataque, sem garantir a ausência de outras falhas.*

O benefício para o usuário — no exemplo, a animação/conteúdo multimídia — pode continuar existindo. Na demonstração, "Reproduzir a mesma função" funcionou nas duas situações; o que mudou foram as

dependências: de "Player legado / Dependência adicional" para "API nativa suportada", com o player marcado como removido. A função foi reimplementada com recursos do próprio navegador.

Remover reduz a superfície de ataque porque sai de cena um programa inteiro que precisava ser instalado, atualizado separadamente em cada máquina e que processava conteúdo vindo de qualquer site. Menos componentes significa menos código exposto e menos atualizações que dependem do usuário.

Isso não garante ausência de falhas: a etiqueta "Aplicação ainda precisa de controles" continua na tela. O código novo pode ter erros próprios, e problemas de entrada, sessão e autorização não dependem do player. Vale notar também a distinção das notas do marco: fim de suporte (deixar de receber correções) e bloqueio de conteúdo (o navegador deixar de executar) são momentos diferentes.

Q30 — Síntese — três mecanismos

> *Pergunta: Compare Formulários, Cookies e DOM em três parágrafos. Em cada um, identifique benefício, entrada, componente responsável, limite de confiança e controle testado. Explique por que os três incidentes não devem ser atribuídos ao mesmo tipo de erro.*

Formulários (INV-01): o benefício é consultar o catálogo por assunto. A entrada é o valor do campo Assunto, que chega como parâmetro da requisição. O componente responsável é a aplicação no servidor, que montava a consulta por concatenação, e o banco, que interpretou o comando alterado. O limite de confiança está entre o dado do cliente e a instrução SQL. O controle que funcionou foi a parametrização, que mantém o valor separado da sintaxe; HTTPS e limite de caracteres não atuaram nesse ponto e desativar a busca eliminou o benefício.

Cookies (INV-02): o benefício é manter a sessão e as reservas reconhecidas entre páginas. A "entrada" aqui não é um texto malicioso, e sim uma requisição inteira iniciada em outro contexto, à qual o navegador anexou a credencial automaticamente. O componente responsável é o servidor, que reconheceu a sessão mas não verificou se a operação era legítima. O limite de confiança está entre "sessão válida" e "pedido intencional do usuário". O controle que funcionou foi o token anti-CSRF validado no servidor; HttpOnly e a troca para POST protegem outras coisas.

DOM (INV-03): o benefício é publicar comentários sem recarregar a página. A entrada é o texto do comentário de um visitante. O componente responsável é o código do cliente, que usava uma operação que interpreta HTML, e o navegador, que criou a estrutura. O limite de confiança está entre o dado não confiável e a API que o insere no documento. O controle que funcionou foi textContent, adequado ao requisito de texto simples.

Não são o mesmo erro porque o ponto de falha, o componente e o controle são diferentes: no primeiro, dado virou comando no servidor/banco; no segundo, não houve dado interpretado, e sim falta de verificação da origem/intenção de uma operação; no terceiro, dado virou estrutura no navegador. O que têm em comum é confiar onde deveria haver uma verificação — mas a correção de um não resolve os outros, como mostraram os retestes com HTTPS.

Q31 — Síntese — novo cenário

> *Pergunta: Um portal recebe comentários por API, usa sessão por cookie, transmite por HTTPS e esconde com CSS o botão de alterar reservas para certos usuários. Aponte duas verificações necessárias no*

> *servidor e uma no cliente. Justifique qual benefício cada uma deve preservar e quais evidências procuraria para avaliá-las.*

Servidor 1 — autorização no endpoint de alterar reservas. O botão escondido por CSS é só apresentação; o servidor precisa conferir, a cada requisição, se aquele usuário tem permissão e se a reserva é dele. Benefício preservado: quem pode continuar alterando suas reservas normalmente. Evidência: enviar a requisição diretamente com a conta de um usuário para quem o botão está oculto e verificar se a resposta é de recusa (403) e se o estado da reserva não muda; depois repetir com um usuário autorizado e confirmar que funciona.

Servidor 2 — verificação anti-CSRF nas operações que alteram estado (alterar reserva, publicar comentário), já que a sessão é por cookie. Benefício preservado: a continuidade da sessão sem pedir login a cada ação. Evidência: enviar a mesma operação sem token e com token inválido e observar a rejeição antes da alteração; confirmar no formulário legítimo que o token existe e é aceito; conferir os atributos do cookie (SameSite, Secure, HttpOnly). Além disso, se os comentários forem gravados em banco, a consulta deve ser parametrizada.

Cliente — inserir os comentários recebidos da API com uma operação de texto (textContent) em vez de interpretá-los como HTML. O fato de virem em JSON e por HTTPS não os torna confiáveis. Benefício preservado: comentários aparecendo na hora, sem recarga. Evidência: ler o código do cliente para localizar a operação de inserção, publicar um comentário com marcação inofensiva e inspecionar o DOM final para confirmar que ela aparece literalmente.

Responsabilidades: o HTTPS protege o trajeto; o cookie identifica a sessão; o CSS organiza a aparência; a API transporta os comentários. Nenhum deles decide acesso ou interpretação — isso cabe ao código do servidor e do cliente.

Q32 — Síntese — evolução da Web

> *Pergunta: Escreva de 200 a 300 palavras conectando hipertexto, comunicação, execução no navegador e segurança. Use pelo menos quatro marcos, incluindo uma defesa ou retirada de componente. Diferencie a data histórica da tecnologia da descoberta de uma vulnerabilidade e conclua sem usar a regra simplista "mais tecnologia = mais insegurança".*

A Web começou, em 1990, como hipertexto: documentos escritos em HTML e ligados entre si, circulando sobre uma Internet que já existia. O usuário apenas lia. Em 1993, os formulários permitiram enviar dados ao servidor, e em 1994 os cookies deram continuidade entre páginas. A comunicação passou a ter duas direções e memória, e com isso o servidor precisou decidir o que fazer com o que recebe e em quem confiar.

Em 1995, o HTTPS protegeu o trajeto, e o JavaScript, seguido pelo DOM em 1998, levou processamento para dentro do navegador. A página deixou de ser um documento fixo e virou um programa que se altera. Applets Java e Flash ampliaram essa execução com componentes adicionais, que cada usuário precisava manter.

As três investigações mostraram que os incidentes não nasceram dessas tecnologias, mas da forma como a aplicação ligava entradas a pontos de interpretação: uma consulta concatenada, uma operação aceita só pela sessão, um comentário inserido como marcação. Por isso as datas dos marcos indicam quando a tecnologia surgiu ou foi padronizada, não quando injeção de SQL, CSRF ou XSS foram descobertos; essas falhas foram descritas depois.

A evolução também trouxe defesas e retiradas. O WebAuthn, em 2019, substituiu o segredo reutilizável por uma prova com chave pública, e o fim do Flash, em 2020, manteve a função com recursos nativos e eliminou um componente inteiro.

Minha conclusão é que cada capacidade nova cria um ponto em que alguém precisa verificar algo. A segurança depende de colocar o controle certo na camada certa, preservando o benefício, e não da quantidade de tecnologia.

ÍNDICE DE EVIDÊNCIAS

Uma captura pode ser substituída por uma transcrição objetiva da tela acompanhada do caminho percorrido. Não use capturas de exemplo como se fossem suas.

[Salve as capturas na pasta Evidencias com os nomes abaixo. As descrições seguem o que o laboratório exibe; confira com a sua tela.]

E01 — Formulários: situação inesperada / descrição:

[captura E01 ainda não salva em Evidencias/]

Arquivo: Evidencias/E01_Formularios_incidente.png. Marco 1993 — Imagens e formulários · modo Benefícios · Com o recurso · sem intervenção. Sequência: busquei "redes", "web" e "http"; depois usei Dinâmica → Disparar agora e aguardei a sequência terminar. Tela: bloco "Resultado inesperado · cadastros fictícios" com Ana · leitor 014 (ana@aluno.invalid), Bruno · leitor 027 (bruno@aluno.invalid) e Carla · leitor 039 (carla@aluno.invalid), e o aviso "Por que a busca mostrou cadastros?". Sustenta Q04. Permite concluir que a resposta saiu do contrato da consulta pública (títulos e autores); não permite, sozinha, dizer a causa.

E02 — Formulários: reteste e uso legítimo / descrição:

[captura E02 ainda não salva em Evidencias/]

Arquivo: Evidencias/E02_Formularios_reteste.png. Mesmo marco · Investigar este marco → 4 Proteger → "Parametrizar a consulta no servidor" → Aplicar ao modelo e testar os dois usos → Ver o resultado e usar a aplicação → Testar uso legítimo. Tela: cartões "Efeito impedido no modelo" e "Benefício preservado"; na demonstração, "Efeito contido; benefício preservado" e a busca por "redes" retornando 1 material (Redes de computadores). O histórico de tentativas lista as quatro intervenções. Sustenta Q06 e Q07. Permite concluir que, nas condições do modelo, o controle contém o efeito mantendo a busca.

E03 — Cookies: situação inesperada / descrição:

[captura E03 ainda não salva em Evidencias/]

Arquivo: Evidencias/E03_Cookies_incidente.png. Marco 1994 — Cookies · Benefícios · Com o recurso · sem intervenção. Sequência: reservei um material, fui a outra página, usei Dinâmica → Disparar agora. Tela: local de retirada "Balcão externo", selo "Alteração inesperada", comparação "Você escolheu: Biblioteca central / Servidor registrou: Balcão externo", sessão fictícia visita-A mantida, aviso "Você pediu para mudar o local de retirada?". Sustenta Q09. Permite concluir que o estado mudou com a sessão ainda válida; não mostra, sozinha, de onde partiu a operação.

E04 — Cookies: reteste e uso legítimo / descrição:

[captura E04 ainda não salva em Evidencias/]

Arquivo: Evidencias/E04_Cookies_reteste.png. Mesmo marco · Proteger → "Validar token anti-CSRF no servidor" → Aplicar e testar → Ver o resultado → Testar uso legítimo. Tela: "Efeito impedido no modelo" (requisição rejeitada com 403; Biblioteca central preservada) e "Benefício preservado" (alteração legítima com token válido continua funcionando); local de retirada de volta a Biblioteca central. Sustenta Q11 e Q12. Parte da validação do uso legítimo é apresentada como resultado do modelo, não por um formulário de alteração.

E05 — DOM: situação inesperada / descrição:

[captura E05 ainda não salva em Evidencias/]

Arquivo: Evidencias/E05_DOM_incidente.png. Marco 1998 — DOM e documentos dinâmicos · Benefícios · Com o recurso · sem intervenção. Sequência: publiquei dois comentários, usei Dinâmica → Disparar agora. Tela: título "Quem está falando nesta página?", comentário "Entrada de visitante no modelo — AVISO NÃO OFICIAL" e faixa "ESTE AVISO NÃO VEIO DA BIBLIOTECA — Uma entrada ganhou aparência de interface no modelo". Sustenta Q19. Permite concluir interpretação de marcação; não comprova execução de script.

E06 — DOM: reteste e novo comentário / descrição:

[captura E06 ainda não salva em Evidencias/]

Arquivo: Evidencias/E06_DOM_reteste.png. Mesmo marco · Proteger → "Inserir o comentário com textContent" → Aplicar e testar → Ver o resultado e usar a aplicação → publiquei um novo comentário. Tela: "Efeito contido; benefício preservado", "A mesma entrada, agora literal: AVISO NÃO OFICIAL" e o meu novo comentário no mural, sem recarga. Sustenta Q21 e Q22. Permite concluir que, para o requisito de texto simples, o controle mantém a entrada como dado e preserva a participação.

LIMITAÇÕES DE EXECUÇÃO

Etapa ou recurso que não funcionou: [PREENCHER — ou "Nenhuma: todas as etapas foram executadas."]

Sintoma e tentativa de solução: [PREENCHER se houver]

O que foi observado / o que ficou sem teste: Nos três marcos investigáveis usei Dinâmica → Disparar agora em vez de aguardar os 120 segundos. A extensão opcional "Simular um possível efeito de XSS" [foi / não foi] executada. O DVWA não foi utilizado (não é exigido).

ARQUIVOS EXPORTADOS PELA FERRAMENTA

Ficha legível (.md): Ficha_Evolucao_WEB_2026-10-01.md [confirmar o nome do arquivo baixado]

Registros (.json), se exportados: [PREENCHER ou "não exportado"]

CONFERÊNCIA FINAL

[ ] Q01–Q32 respondidas, ou limitação concreta registrada.

[ ] Quatro intervenções comparadas em cada investigação.

[ ] E01–E06 identificadas.

[ ] Hipóteses iniciais preservadas e revisões justificadas.

[ ] Diferenciei observações do modelo de evidências de um sistema real.

[ ] Salvei o documento principal e a ficha exportada.

Não existe envio automático a Notion, professor ou plataforma de avaliação. Este arquivo é o produto principal da atividade. Preserve uma cópia local.